Windowed Off-Policy Expert Value-Agreement

A cheap off-policy diagnostic: does a policy match the expert's windowed value from expert states?

Purpose and scope

On a past task i, this measures whether a policy achieves the same windowed value as that task's expert, starting from states the expert actually reached. It answers can the new model play as well as the expert? without paying for many full-episode rollouts and without letting easy early-game play inflate the score. It is NOT a standalone forgetting metric: it conditions on expert-reached states and says nothing about whether the policy can REACH them on its own. It is reported ALONGSIDE the on-policy no-op anchor (greedy-100), never instead of it. VERIFICATION: the implementation PASSED code-verifier against the design doc.

Value scale and snapshot environment

Shared by both procedures: discount γ and sign-clip flag (= task.clip_rewards). $\mathrm{clip}(r) = \mathrm{sign}(r) \in \{-1, 0, +1\}$ if clipping else r (matches the training value scale). The windowed value over n steps is $V_\mathrm{win}=\sum_{j=0}^{n-1}\gamma^j\,\mathrm{clip}(r_j)$; the raw window score is $\sum r_j$. Greedy action $=\arg\max$ over the head's logits. The SNAPSHOT ENVIRONMENT (make_snapshot_env) is preprocessing-only: base ALE (frameskip 1, full 18-action set) wrapped by AtariPreprocessing (skip / max-pool, grayscale, resize to 84×84), emitting a SINGLE frame. It has NO frame-stack, NO TimeLimit, NO reward clip, and terminal-on-life-loss forced OFF, so the evaluator manages the 4-frame stack itself and a restored mid-episode state is not spuriously terminated. It requires sticky-actions off for bit-exact clone/restore.

Procedure 1: build the expert state set (once per task, reusable)

Precomputed ONCE per task and reused across every checkpoint and method (the dominant cost saving). A window is CONSEQUENTIAL iff its raw score $\geq$ consequential-thresh (expert scored non-trivially) AND its discounted-clipped value $V^*_\mathrm{win} \geq v_\mathrm{floor}$ (so the relative-shortfall denominator is bounded away from 0). The adaptive horizon H_i is the smallest grid H with $\geq \tau$ consequential windows; this and the consequential filter

together avoid the $0 \approx 0$ vacuity of sparse Atari games.

Algorithm 1: build-expert-state-set(expert, task)

Input: expert policy π_i^* (greedy source of states); task i (game, γ, frame-stack fs, clip)

Input: grid $G = (15, 20, 25, 30, 40, 60)$; $\tau = 0.5$; consequential-thresh $= 1$; $v_{\text{floor}} = 0.1$

Input: $n_{\text{starts}} = 20$ no-op starts; cap $N = 5000$ kept states; rng seed

Output: ExpertStateSet: game, task-id, horizon H_i, γ, fs, clip, list of ExpertState, meta

Output: each ExpertState: ale-state, stacked-obs, V^*_{win}, raw-win, n_{win}, steps-to-term, category (pickled)

(A) ROLL the expert and snapshot every visited state:

  for $k = 1, \ldots, n_{\text{starts}}$ **do**

    env $\leftarrow$ snapshot-env; frame $\leftarrow$ env.reset(seed $+ k$) (no-op start diversity)

    stack $\leftarrow$ deque of fs copies of the first frame (frame-stack reset semantics)

    while not done and steps $<$ cap **do**

      obs $\leftarrow$ stack as $(\text{fs}, 84, 84)$ oldest..newest

      $a \leftarrow$ greedy expert action on obs; $s_t \leftarrow$ ale.cloneState() (SNAPSHOT BEFORE steppin

      frame, r, term, trunc $\leftarrow$ env.step(a); record $(s_t,\ \text{obs},\ r)$; push frame

    record whether the trajectory TRULY terminated (vs hit the step cap)

(B) ADAPTIVE horizon: window(i, t, H) caps at steps-to-terminal, returns $(V^*_{\text{win}},\ \text{raw},\ n_{\text{win}})$

  consequential$(V^*_{\text{win}}, \text{raw}) \equiv (\text{raw} \geq \text{thresh})$ and $(V^*_{\text{win}} \geq v_{\text{floor}})$

  $\text{frac}(H) \leftarrow$ fraction of ALL (i, t) starts whose H-window is consequential

  $H_i \leftarrow$ smallest $H \in G$ with $\text{frac}(H) \geq \tau$ (**else** max$G$, record a warning)

(C) COLLECT consequential candidates at H_i, **then** sample:

  for each trajectory i and start t: $(V^*_{\text{win}}, \text{raw}, n_{\text{win}}) \leftarrow$ window(i, t, H_i)

    if not consequential: skip (keep ONLY consequential states)

17 category $\leftarrow$ near-terminal **if** (truly-terminated and steps-to-term $\leq H_i$), **else** early **if** $t < 20$, o

18 append ExpertState(s_t, obs$_t$, V^*_{win}, raw, n_{win}, steps_to_term, category)

19 **if** kept $> N$: keep a uniform random subset of size N (no replacement); pickle the set

---

REMARK: cloneState / restoreState are O(1), replacing seed + action-prefix replay that would cost an entire episode for a near-terminal state. The snapshot is taken BEFORE env.step, so a stored s_t pairs with the obs the expert acted on and the reward r_t that action produced. The frame stack is reconstructed exactly from the stored obs; it is never snapshotted through the ALE core.

Procedure 2: evaluate expert agreement (per checkpoint, per task)

Restores each expert state (O(1)) and rolls the POLICY greedily for that state's stored n_{win} steps (the SAME window length the expert's V^*_{win} covered, so the returns are strictly equal-horizon, capped at terminal). Reports the ONE-SIDED shortfall: the expert is the ceiling, so beating it is not penalized (hinged at 0), mirroring the training hinge $[\hat{V}^L_k - \hat{V}^G_k]_+$ of Part A. States are processed in banks of n_{envs} snapshot envs run synchronously.

Algorithm 2: evaluate-expert-agreement(policy, task, state-set, task-id)

Input: policy; task-id (the head to evaluate); precomputed state-set (supplies $H, \gamma,$ fs, clip)

Input: $n_{\text{envs}} = 8$ snapshot envs as a synchronous bank

Output: agreement-gap $= \text{mean}_s\,[V^*_{\text{win}}(s) - V^\pi_{\text{win}}(s)]_+$ (raw units)

Output: relative-gap $= \text{mean}_s\,[\cdot]_+ / V^*_{\text{win}}(s) \in [0,1]$; n-states; horizon; per-category relative-gap

Output: lower is better; 0 $\Rightarrow$ policy matches or beats the expert on every kept state

1 **if** state-set empty: **return** NaN gaps, n-states $= 0$

2 create n_{envs} snapshot envs; reset each once (seed 0) to init ALE before any restore

3 **for each** chunk of up to $B = n_{\text{envs}}$ states **do**

4 **for** lane j, expert state s: ale$_j$.restoreState(s); sync stale life counter; seed stack$_j$ from stored o

5 $V^\pi_j \leftarrow 0$; $g_j \leftarrow 1$; done$_j \leftarrow$ false; window length $w_j \leftarrow s.n_{\text{win}}$

for step $= 0, \ldots, \max_j w_j - 1$ (**break if** all done) **do**

  acts $\leftarrow$ batched greedy POLICY actions on stacks at task-id

  for each active lane j:

    if step $\geq w_j$: mark done$_j$ (reached the expert window length); **continue**

    frame, r, term, trunc $\leftarrow$ env$_j$.step(acts$_j$)

    $V_j^\pi \leftarrow V_j^\pi + g_j \cdot \mathrm{clip}(r)$; $g_j \leftarrow g_j \cdot \gamma$; slide stack$_j$ (drop oldest, append frame)

    if term or trunc: mark done$_j$ (cap at terminal)

for lane j, expert state s: gap $\leftarrow \max(0,\ s.V^*_{\mathrm{win}} - V_j^\pi)$; rel $\leftarrow$ gap$/s.V^*_{\mathrm{win}}$

  record gap, rel, and rel under s.category

return mean gap, mean rel, n-states, H, per-category mean rel